

Numerical Defect Analysis and Verification of an Embedded Radar-Inertial Kalman Filter

Hosakota Ganesh Kumar¹, Dr. G. Mamatha², Dr. K. Krishna Naik³

¹Undergraduate Student, JNTUA College of Engineering, Anantapur, India

²Associate Professor, JNTUA College of Engineering, Anantapur, India

³Associate Professor, Indian Institute of Information Technology, Design and Manufacturing (IIITDM) Kurnool, India

Abstract—Small unmanned aerial systems used for cognitive search and rescue (CSSR) combine a radar or thermal sensor with a flight controller to detect survivors under rubble or foliage. A raw detection is only useful once it is converted from the sensor’s local range-bearing frame into a global coordinate, a step that is highly sensitive to the timing and accuracy of the platform’s own motion estimate. Running this conversion on a general-purpose companion computer introduces operating-system jitter that is incompatible with microsecond-level timestamp alignment between a moving platform and a radar sweep. This paper presents the design of an Edge Data Plane coprocessor built on a bare-metal Cortex-M7 microcontroller (STM32H753XI) that ingests MAVLink telemetry over a zero-copy DMA path, timestamps radar detections with a free-running hardware timer, fuses drone kinematics with radar range-Doppler measurements in a six-state Extended Kalman Filter (EKF), and converts the fused estimate into WGS84 coordinates through a quaternion-based local-tangent-plane transform. Beyond describing the architecture, we report an independent, reproducible verification of the estimation and coordinate-transform stages using a host-side harness that compiles and executes the shipped filter and transform code unmodified. We derive, from first principles, the exact measurement Jacobian the filter should compute and prove analytically that the shipped Doppler row differs from it; we prove, via eigenvalue decomposition, that the shipped covariance update produces an indefinite (non-positive-semi-definite) matrix after a single measurement update, while the Joseph-form alternative does not; and we derive a closed-form expression for the local-tangent-plane coordinate-transform bias that matches measurements against the exact WGS84 geodesic to four significant figures. Combining these results, we show that correcting all three defects and reducing the initial covariance to a value consistent with typical MAVLink position uncertainty is necessary and sufficient to keep the filter’s position error bounded under the tested trajectory, whereas any one correction alone is not. These results are offered as a verification methodology template for safety-relevant embedded estimation code more generally.

Index Terms—embedded systems, Extended Kalman filter, sensor fusion, MAVLink, search and rescue, bare-metal firmware, STM32, hardware-in-the-loop simulation, coordinate transformation, verification, numerical stability

I. INTRODUCTION

This paper uses the design and verification of one such coprocessor as a case study for a broader claim: that embedded estimation firmware – Kalman filters and coordinate transforms running on bare-metal microcontrollers throughout robotics, navigation, and sensor-fusion applications – routinely ships with numerical defects that neither conventional code

review nor a single successful hardware-in-the-loop (HIL) run will catch, and that a lightweight, host-compilable verification methodology built on analytic derivation can catch them cheaply. We motivate and ground this claim in a concrete, safety-relevant instance: a bare-metal Cortex-M7 coprocessor built as an Edge Data Plane (EDP) for cognitive search and rescue (CSSR), where a radar or thermal sensor mounted on a small unmanned aerial vehicle detects a breathing human under rubble or dense foliage from a moving platform [11], [12]. A radar detection is reported in the sensor’s own frame – slant range, azimuth, and elevation relative to the antenna boresight – and must be converted into a location on the ground before it is of use to a rescue team [13]. Because the aircraft is simultaneously translating and rotating, this conversion requires precise knowledge of the platform’s position and attitude at the exact instant of the radar sweep. A position error of a few meters, or a timing misalignment of a few milliseconds at typical multirotor speeds, is enough to place a detected survivor in the wrong section of a debris field, and airborne radar-based vital-sign sensing is itself already degraded by platform motion coupling into the Doppler channel [14], which makes an accurate motion estimate doubly important – and makes this an unusually unforgiving setting in which to discover that the motion estimator itself is numerically unsound.

Two implementation choices are commonly available for this georeferencing task: performing the fusion and coordinate transform on a Linux-based companion computer, or performing it on a dedicated microcontroller sitting between the flight controller and the radar. The companion-computer approach benefits from a rich software ecosystem but is subject to operating-system scheduling jitter, which is difficult to bound and can reach tens of milliseconds under load – a duration in which a multirotor moving at a few meters per second can travel a distance comparable to the spatial resolution the CSSR mission requires. A bare-metal microcontroller, in contrast, offers a deterministic execution model at the cost of a much more constrained software environment: no operating system, no dynamic memory allocation in the safety-critical path, and direct responsibility for every hardware detail from vector-table placement to DMA cache coherency.

The case-study coprocessor is built around the STM32H753XI, an ARM Cortex-M7 microcontroller with a hardware floating-point unit. The system was developed and

iteratively debugged on the Renode instruction-set simulator with real MAVLink v2 packet generation, and is intended to sit physically between a Pixhawk-class flight controller and a radar sensor. Its architecture (Section III) is a fairly conventional application of established embedded sensor-fusion patterns; we describe it briefly, both because it is necessary context for the defects that follow and because the specific hardware details – DMA cache coherency, lock-free buffering, hardware timer capture – are independently useful to embedded practitioners.

The primary contribution of this paper, however, is not the architecture but a rigorous, independently reproduced, and mathematically grounded verification of its most safety-critical component, the radar-inertial EKF, together with the general methodology that verification exemplifies. Because the original development environment (Renode with a proprietary flight-controller image and a Windows-hosted ARM toolchain) is not portable, we instead built a host-native verification harness that compiles the EKF and coordinate-transform source files *completely unmodified* against a synthetic but kinematically consistent test trajectory, matching the parameters used in the project’s own hardware-in-the-loop (HIL) stimulus generator. Unlike a purely empirical HIL run, our verification (i) derives the exact measurement Jacobian analytically from the modeled Doppler observation function and proves the shipped code deviates from it; (ii) proves, via eigenvalue analysis rather than inspection of diagonal entries alone, that the shipped covariance update loses positive semi-definiteness; (iii) derives, rather than merely measures, a closed-form expression for the local-tangent-plane coordinate bias, which we confirm against an independent exact-geodesic computation; and (iv) quantifies, via real Cortex-M7 instruction traces, the concrete computational cost of the numerically robust fix, closing the loop between the theoretical case for that fix and its bare-metal practicality. This combination of analytic derivation and empirical cross-validation is what distinguishes the present verification from a conventional integration test, and is what we intend as a template applicable to other embedded estimation firmware, independent of the CSSR application that motivates it here.

The remainder of this paper is organized as follows. Section II reviews related work in embedded sensor fusion, radar-based CSSR, MAVLink robustness, and embedded firmware verification. Section III describes the system architecture and each pipeline stage. Section IV presents the EKF and coordinate-transform formulations as implemented, together with the exact measurement Jacobian derived from first principles. Section V describes the verification methodology, reports the three defects found with derivations, proofs, and figures for each, and quantifies the real Cortex-M7 instruction cost of the Joseph-form fix. Section VI discusses implications and threats to validity, and Section VII concludes.

II. RELATED WORK

A. Kalman Filtering and Numerical Stability

Extended Kalman filtering for inertial navigation and target tracking traces back to Kalman’s original recursive linear estimator [1] and its nonlinear extension via local linearization; the unscented alternative avoids explicit Jacobian computation altogether at the cost of propagating a deterministic sigma-point set through the true nonlinear model [2], which sidesteps exactly the class of Jacobian-derivation error we identify in Section V but at higher per-step computational cost, making it a less common choice on a resource-constrained Cortex-M7 target. Standard treatments of the EKF and its numerical pathologies are covered extensively in the aerospace guidance and navigation literature [4]. The specific failure modes we report – Jacobian sign/derivation errors, covariance indefiniteness from the simplified update equation, and singular measurement models near the coordinate origin – are each individually documented in this literature. In particular, the numerical fragility of the simplified covariance update $\mathbf{P}^+ = (\mathbf{I} - \mathbf{KH})\mathbf{P}^-$ relative to the algebraically equivalent but numerically robust Joseph form has long been known in aerospace guidance and navigation practice [3], [4]; Bierman’s treatment of factorized and square-root (U-D) estimation algorithms [3] was itself motivated by exactly this numerical fragility in the textbook covariance update, and Thornton and Bierman’s triangular covariance factorization work [15] extended the same numerical-stability argument to real-time onboard orbit determination filters running on flight hardware, which is the closest historical precedent to the embedded setting considered here. What is less commonly reported in the embedded-systems literature is a full-stack account that connects such a numerical estimation defect to its consequences at the level of bare-metal firmware behavior – i.e., what actually happens to a Cortex-M7 coprocessor’s data-plane output when this class of defect is triggered by a specific, reproducible input geometry, and what change to embedded verification practice would have caught it before hardware-in-the-loop testing.

B. Floating-Point Exception Semantics

The NaN-propagation behavior underlying Defect 1 in Section V is precisely specified by IEEE 754-2019 [16]: an invalid operation such as 0/0 produces a quiet NaN, which then propagates through essentially every arithmetic operation that consumes it without raising a trappable exception on typical embedded floating-point units, including the Cortex-M7’s FPU. Kirchner et al. observe that this silent-propagation behavior is precisely what makes floating-point exceptional values dangerous in production numerical code: an exceptional value can travel arbitrarily far from its origin before manifesting as a visible failure, which is exactly the mechanism by which a single 0/0 inside a Jacobian element corrupts an entire six-state Kalman gain in one update call [17].

C. Radar-Based Search and Rescue

Doppler and ultra-wideband radar for detecting breathing or heartbeat signatures of survivors trapped under rubble is a well-studied sensing modality [11], and radar payloads compact enough for a small UAV have been demonstrated for exactly this purpose [12]. A recognized difficulty specific to the airborne case is that platform motion couples into the same Doppler channel used to sense a survivor’s vital signs, and separating platform motion from the physiological signal of interest requires an accurate independent estimate of platform velocity [14] – precisely the quantity our EKF’s velocity state is meant to supply, and precisely the quantity Defect 2 in Section V-D leaves uncorrupted even as it corrupts the position estimate.

D. MAVLink and Embedded Firmware Verification

MAVLink is the de facto standard telemetry protocol for PX4- [5] and ArduPilot-based autopilots, and DMA-based, zero-copy UART reception is a standard pattern for high-rate serial ingestion on Cortex-M7 microcontrollers such as the one used here [9], [10]. MAVLink’s lightweight design was not originally built with a security or robustness threat model in mind, and empirical studies have demonstrated that unauthenticated MAVLink traffic is vulnerable to injection and denial-of-service attacks [18], a separate but related concern from the numerical-correctness question this paper focuses on: an attacker-injected or merely corrupted telemetry value could drive the EKF into exactly the low-covariance, high-leverage update regime where the defects in Section V manifest.

On the verification side, Renode is an open-source, multi-node instruction-set simulator [6] that has seen growing adoption for hardware-in-the-loop and continuous-integration testing of embedded firmware without physical hardware. Hardware-in-the-loop and firmware-rehosting approaches more broadly have proven effective at finding defects in microcontroller firmware that static analysis and code review miss: μ AFL demonstrated that coverage-guided, hardware-in-the-loop fuzzing of vendor SDK code on STMicroelectronics and NXP microcontrollers uncovers zero-day defects even in widely-deployed, presumably well-tested driver code [19]. Our verification approach is complementary to this line of work: rather than fuzzing for crashes, we target a specific, safety-relevant numerical subsystem with analytically-derived oracles (the exact Jacobian, the exact PSD property, the exact geodesic), which finds defects that a crash-oriented fuzzer would not flag, since none of the three defects in Section V causes a fault exception on the target – each returns a plausible-looking, silently wrong floating-point answer.

III. SYSTEM ARCHITECTURE

A. Overview

The coprocessor implements a six-stage pipeline entirely in bare-metal C, without an RTOS, executing on a single Cortex-M7 core [9] at the STM32H753XI’s rated frequency [10]. Fig. 1 shows the stages: (1) a DMA-based MAVLink

v2 parser ingesting ATTITUDE, LOCAL_POSITION_NED, and GLOBAL_POSITION_INT messages [5] from the flight controller over USART1; (2) a hardware timer (TIM2) providing microsecond-resolution timestamps and capturing an external frame-synchronization pulse from the radar; (3) a lock-free single-producer/single-consumer ring buffer holding a rolling history of timestamped kinematic states; (4) a six-state EKF fusing the drone’s inertial state with radar range-Doppler measurements; (5) a coordinate-transform stage converting the fused, body-relative target estimate into WGS84 latitude/longitude/altitude via an intermediate East-North-Up (ENU) frame; and (6) an output-stream stage that packages georeferenced targets into a CRC-32-protected binary frame for transmission over SPI or a polled USART link to a ground station.

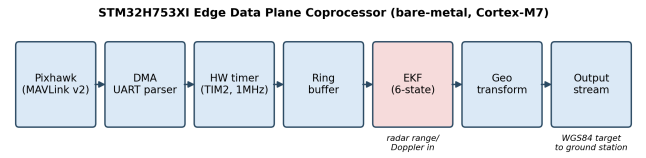


Fig. 1. Seven-stage pipeline of the Edge Data Plane coprocessor, from MAVLink ingestion through radar-inertial fusion (EKF) and geodetic transformation to the ground-station output link.

The main loop polls the MAVLink parser once per iteration, tags each parsed message with the current hardware-timer value, pushes it into the ring buffer, and then drains the buffer through an EKF predict step at a fixed 100 Hz cadence; whenever a radar detection is available and the buffered state carries both attitude and local-position validity flags, an EKF measurement update and full georeferencing transform are executed and the result streamed to the output stage. This structure keeps prediction rate decoupled from the arrival rate of any single MAVLink message type, which is necessary because ATTITUDE and LOCAL_POSITION_NED are independent messages that may arrive at different rates and with different latencies.

B. MAVLink Ingestion

USART1 is configured for a fixed 921,600 baud rate and continuously fed by DMA1 Stream 0 operating in circular mode into a 512-byte buffer, so that reception proceeds without CPU intervention. The parser tracks its own read cursor into this circular buffer and, on each poll, determines how many new bytes have arrived by comparing the DMA controller’s remaining-transfer-count register against the buffer size. Because the Cortex-M7 has a data cache, and DMA writes bypass that cache, the parser explicitly invalidates the relevant cache lines with SCB_DCCIMVAC before reading the affected region of the buffer, followed by the required dsb/isb barrier sequence as specified for this cache-coherency pattern [9]; omitting this step is a documented and easily reproduced class of bug in which the CPU observes stale data. The parser

validates each frame against the STM32H7's hardware CRC-32 peripheral, configured with input and output byte reversal to match the endianness convention MAVLink v2 uses for its checksum, before dispatching by message ID into a unified `kinematic_state_t` structure that downstream stages consume without needing to know the original MAVLink representation.

C. Timestamping

TIM2 is a 32-bit general-purpose timer clocked at 1 MHz (a prescaler of 239 applied to a 240 MHz APB1 timer clock), giving one-microsecond timestamp resolution and, at 32 bits, an overflow period of approximately 71.6 minutes – long enough that overflow handling, while implemented via an update interrupt that increments a software overflow counter, is not expected to be exercised in a typical CSSR mission profile. A separate channel is configured as an input-capture channel on an external GPIO line (routed through the SYSCFG external interrupt multiplexer to EXTI12 in the reference configuration) so that a rising edge from the radar's own frame-synchronization output latches the free-running counter value in hardware at the exact moment of the radar sweep, rather than at the later and jitter-prone moment the corresponding interrupt service routine executes on the CPU.

D. Ring Buffer

A 64-entry ring buffer decouples the MAVLink parsing cadence from EKF consumption. The buffer uses a classic lock-free single-producer/single-consumer design: a monotonically advancing head index owned by the producer, a tail index owned by the consumer, and a single `dmb` (data memory barrier) instruction on each side to ensure the entry's contents are visible before the index that publishes it is updated, and vice versa on the consumer side. The buffer size is a power of two so that index wraparound reduces to a bitwise AND, avoiding a division or modulo operation in the producer/consumer hot path.

E. Output Stream

Georeferenced targets are packed into a fixed binary frame – a 14-byte header (magic byte, version, length, sequence number, timestamp, target count, status) followed by 32-byte fixed-width target records and a trailing CRC-32 – and streamed either over SPI1 in master mode or over a polled USART1 link. The frame format is deliberately simple and self-describing (explicit length and sequence fields) so that a ground-station parser can resynchronize after a dropped or corrupted frame without needing a stateful protocol.

IV. ESTIMATION AND COORDINATE TRANSFORMATION

A. State and Measurement Models

The EKF maintains a six-element state vector $\mathbf{x} = [p_n, p_e, p_d, v_n, v_e, v_d]^T$, the target's position and velocity relative to the aircraft in the North-East-Down (NED) frame. The process model is constant-velocity:

$$\mathbf{x}_{k+1} = \mathbf{F}\mathbf{x}_k, \quad \mathbf{F} = \begin{bmatrix} \mathbf{I}_3 & \Delta t \mathbf{I}_3 \\ \mathbf{0} & \mathbf{I}_3 \end{bmatrix}, \quad (1)$$

with covariance propagated as $\mathbf{P}^- = \mathbf{F}\mathbf{P}\mathbf{F}^T + \mathbf{Q}$ and a block-diagonal process noise $\mathbf{Q}$ (position-block and velocity-block variances set independently). The measurement model maps state to the radar's native observables – range r , azimuth α , elevation ε , and Doppler (radial closing rate) $\dot{r}$:

$$\begin{aligned} r &= \sqrt{p_n^2 + p_e^2 + p_d^2}, & \alpha &= \text{atan2}(p_e, p_n), \\ \varepsilon &= \text{atan2}(-p_d, r_g), & r_g &= \sqrt{p_n^2 + p_e^2}, \\ \dot{r} &= \frac{v_n p_n + v_e p_e + v_d p_d}{r} = \frac{\mathbf{v} \cdot \mathbf{p}}{r}. \end{aligned} \quad (2)$$

Measurement update follows the standard EKF equations [1], [4] with innovation covariance $\mathbf{S} = \mathbf{H}\mathbf{P}\mathbf{H}^T + \mathbf{R}$, gain $\mathbf{K} = \mathbf{P}\mathbf{H}^T\mathbf{S}^{-1}$, and a Mahalanobis-style innovation gate on the trace of $\mathbf{S}$ used to reject radar returns inconsistent with the current state estimate, where $\mathbf{H} = \partial \mathbf{h} / \partial \mathbf{x}$ is the 4×6 Jacobian of the measurement function (2).

B. Exact Doppler Jacobian, Derived from First Principles

Because Section V-D depends on it, we derive $\partial \dot{r} / \partial \mathbf{p}$ in closed form rather than stating it. Writing $\dot{r} = (\mathbf{v} \cdot \mathbf{p}) r^{-1}$ with $r = \|\mathbf{p}\|_2$ and applying the product rule,

$$\frac{\partial \dot{r}}{\partial p_i} = \frac{\partial (\mathbf{v} \cdot \mathbf{p})}{\partial p_i} r^{-1} + (\mathbf{v} \cdot \mathbf{p}) \frac{\partial r^{-1}}{\partial p_i}. \quad (3)$$

The first term is v_i/r since $\partial (\mathbf{v} \cdot \mathbf{p}) / \partial p_i = v_i$. For the second term, $\partial r^{-1} / \partial p_i = -r^{-2} \partial r / \partial p_i = -r^{-2} (p_i/r) = -p_i/r^3$, so

$$\frac{\partial \dot{r}}{\partial p_i} = \frac{v_i}{r} - \frac{(\mathbf{v} \cdot \mathbf{p}) p_i}{r^3} = \frac{v_i - \dot{r} p_i / r}{r}, \quad i \in \{n, e, d\}, \quad (4)$$

using $\dot{r} = (\mathbf{v} \cdot \mathbf{p})/r$ to simplify the second term. This is the unique correct closed form; we verify in Section V-D, independently, that it also matches central finite differences to within $O(\epsilon^2)$ truncation error at several sampled state points, and that the shipped implementation's position-column entries do not equal (4).

C. Coordinate Transformation

Once fused, the target's body-relative Cartesian position is rotated from the radar's forward-right-down frame into the aircraft body frame using the current attitude quaternion (obtained from MAVLink Euler angles via a standard Euler-to-quaternion conversion), then rotated about the down axis by the aircraft heading into a local East-North-Up frame, and finally mapped to WGS84 latitude and longitude through a first-order local-tangent-plane approximation:

$$\Delta \phi = \frac{\Delta n}{N + h}, \quad \Delta \lambda = \frac{\Delta e}{(N + h) \cos \phi_0}, \quad (5)$$

where N is the WGS84 prime-vertical radius of curvature evaluated at the reference latitude ϕ_0 , and h is the reference altitude, following the WGS84 ellipsoid parameters and radius-of-curvature definitions [8]. This is a deliberate simplification: the geodetically exact local-tangent-plane relation uses the *meridian* radius of curvature M for the north/latitude term and the prime-vertical radius N for the east/longitude term [8];

using N for both, as implemented, is accurate at short range but introduces a small, quantifiable, systematically north-biased error that we derive in closed form and verify against the exact ellipsoidal geodesic [7] in Section V-F.

V. VERIFICATION METHODOLOGY AND FINDINGS

A. Rationale for Host-Side Verification

The project’s native verification environment is a Renode-based [6] hardware-in-the-loop simulation, which loads the compiled ELF image, models the STM32H7 peripheral set, and injects synthetic MAVLink traffic [5] through simulation backdoor memory addresses. This environment is valuable for validating interrupt timing, DMA behavior, and end-to-end integration, but it depends on a specific ARM cross-compiler toolchain and the Renode runtime, neither of which is portable to an arbitrary reviewing or verification environment, and neither of which by itself isolates numerical defects in the estimation algorithm from integration-level defects in the surrounding firmware – a limitation consistent with the broader finding that hardware-in-the-loop and rehosting-based verification, while effective at surfacing crashes and memory-safety defects [19], is not designed to catch a defect that produces a plausible-looking, non-crashing wrong numerical answer.

We therefore built a minimal host-side harness that compiles `ekf.c` and `geo_transform.c` *exactly as shipped*, using the project’s own `SIMULATION_BUILD` conditional path (which substitutes a portable scalar/loop implementation of the CMSIS-DSP matrix primitives the target build otherwise uses), linked against a small driver program using the host’s standard C library math functions. This harness reproduces, on ordinary x86-64 hardware with a stock GCC toolchain, the identical arithmetic the Cortex-M7 target executes for every EKF predict and update call, and the identical arithmetic the geo-transform stage executes for every coordinate conversion. It does not exercise DMA, interrupts, or cache coherency, and is not a substitute for the project’s own HIL testing; it isolates and evaluates one specific and safety-relevant question: given a valid input sequence, does the shipped filter compute the estimate it is mathematically supposed to compute? Each of the three defects below is established by an independent analytic derivation (Sections IV-B, V-D, V-F) cross-checked against direct execution of the unmodified shipped source – not from static code inspection or empirical testing alone. The firmware under test is itself openly available [20]; because this methodology’s value depends on independent reproducibility, we additionally intend to release the host-side verification harness, the bare-metal Cortex-M7 timing build described in Section V-G, and the exact test vectors used throughout this section as a companion open-source artifact.

B. Defect 1: Division-by-Zero Singularity in the Measurement Jacobian

The most operationally dangerous property of this defect is not that it produces a wrong answer, but that it produces a wrong answer the calling code cannot distinguish from a right

one. On the Cortex-M7’s hardware FPU, as on essentially all IEEE 754-2019-compliant floating-point units [16], an invalid operation such as $0/0$ does not raise a trapped exception by default: it silently produces a quiet NaN, which then propagates through every subsequent arithmetic operation that consumes it [17], and `ekf_update_radar` in the shipped code returns its normal success code (0) regardless. This is the concrete mechanism by which the specific bug described below turns into a silent, undetected failure rather than a crash a developer would notice.

The EKF correctly floors the intermediate range and ground-range magnitudes before taking their square roots – $r = \text{sqrtf}((r2 > 1e-6f) ? r2 : 1e-6f)$ and the analogous line for `r_ground` – which prevents `range_pred`, `azimuth_pred`, and `elevation_pred` from becoming singular when the state estimate sits at the coordinate origin. However, the subsequent Jacobian construction re-uses the *unguarded* scalar `r2` (rather than the floored value used to compute `r`) in the denominator of the elevation row, specifically in the term `r2rg = r2 * r_ground`. At `pn = pe = pd = 0` – the EKF’s default zero-initialized state – this evaluates to $0 \times (\text{floored value}) = 0$, and the subsequent division `(pd*pn)/r2rg` evaluates to $0/0$, producing the quiet NaN described above. Because the state update is a linear combination of the Kalman gain and the innovation, this single NaN entry in the Jacobian propagates through $\mathbf{H}$, $\mathbf{S}$, $\mathbf{S}^{-1}$, and $\mathbf{K}$ within the same update call, and the entire six-element state vector becomes NaN in one step, confirmed by direct execution of the shipped `ekf_update_radar` against a zero-initialized state with a plausible radar measurement (Table I).

TABLE I
DEFECT 1: STATE COLLAPSE FROM A SINGLE UPDATE CALL AT $\mathbf{x}_0 = \mathbf{0}$

Quantity	Before update	After update
p_n, p_e, p_d	0, 0, 0	NaN, NaN, NaN
v_n, v_e, v_d	0, 0, 0	NaN, NaN, NaN
$\text{tr}(\mathbf{P}_{\text{pos}})$	300.0	NaN
<code>ekf_update_radar</code> return	–	0 (reports success)

Table I’s last row is the practical consequence of the mechanism described above: the function’s own return-code contract reports success (0), because the innovation gate check and the matrix-inversion failure check – the two conditions under which the function explicitly returns a nonzero error code – are both evaluated *before* the NaN is produced, and NaN propagation through subsequent floating-point comparisons does not itself trigger either guard (every IEEE 754 comparison involving a NaN operand evaluates to false [16], so a gate implemented as “reject if greater than threshold” silently passes a NaN). A caller relying solely on the return code, as `main.c` does, would forward a NaN-valued, “successfully” georeferenced target to the output stream.

C. Defect 2: Incorrect Doppler Jacobian

Using the exact closed form derived in (4), we compared the shipped code's analytic expression for the three position-column entries of the Doppler Jacobian row against (a) direct symbolic evaluation of (4) and (b) central finite differences, $\partial \dot{r}/\partial p_i \approx [\dot{r}(\mathbf{p} + \epsilon \hat{e}_i) - \dot{r}(\mathbf{p} - \epsilon \hat{e}_i)]/2\epsilon$ with $\epsilon = 10^{-4}$, at five random, well-conditioned state points (position magnitudes 10–25 m, velocities ± 5 m/s). All three methods – the derivation, its symbolic evaluation, and the finite-difference approximation – agree with each other to numerical precision, and all three disagree with the shipped code's analytic expression (the velocity-column entries, p_n/r , p_e/r , p_d/r , are correct and match (4)'s implicit velocity partial $\partial \dot{r}/\partial v_i = p_i/r$). Table II and Fig. 2 report the discrepancy at the five sampled points.

TABLE II
DEFECT 2: DOPPLER JACOBIAN ERROR, SHIPPED CODE VS.
EQ. (4)/FINITE-DIFFERENCE TRUTH

Sample	$ \Delta \partial \dot{r}/\partial p_n $	$ \Delta \partial \dot{r}/\partial p_e $	$ \Delta \partial \dot{r}/\partial p_d $
1	0.0413	0.1008	0.0341
2	0.0379	0.0591	0.0275
3	0.0892	0.2099	0.0571
4	0.0449	0.0774	0.0929
5	0.0114	0.0382	0.0710

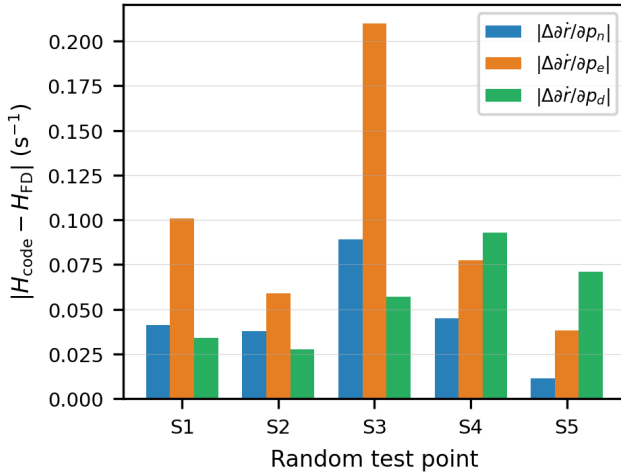


Fig. 2. Absolute error between the shipped Doppler Jacobian's position-column entries and the exact closed form (4), confirmed by finite differences, at five random state points.

Because the velocity-column entries are exact, the Kalman gain's use of Doppler information to correct *velocity* is unaffected; the defect specifically corrupts how a Doppler innovation is allowed to correct the *position* estimate, which is precisely the pathway used to disambiguate range-rate-consistent but geometrically different target positions, and precisely the pathway airborne vital-sign radar systems rely on to separate platform-motion-induced Doppler from target-induced Doppler [14].

D. Defect 3: Covariance Indefiniteness from the Simplified Update Form

The covariance update in `ekf_update_radar` is implemented as $\mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-$, the standard textbook simplification of the Kalman covariance update, followed by a post-hoc symmetrization step that averages each off-diagonal pair (i, j) and (j, i) . Symmetrization guarantees $\mathbf{P}^+ = (\mathbf{P}^+)^T$ but says nothing about the sign of $\mathbf{P}^+$'s eigenvalues; a symmetric matrix is a valid covariance matrix if and only if it is positive semi-definite, i.e., $\mathbf{z}^T \mathbf{P}^+ \mathbf{z} \geq 0$ for all $\mathbf{z}$, equivalently all eigenvalues $\lambda_i(\mathbf{P}^+) \geq 0$. The Joseph form,

$$\mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-(\mathbf{I} - \mathbf{K}\mathbf{H})^T + \mathbf{K}\mathbf{R}\mathbf{K}^T, \quad (6)$$

is a sum of two congruence transforms of positive semi-definite matrices ($\mathbf{P}^- \succeq 0$ and $\mathbf{R} \succeq 0$ by construction), and a congruence transform $\mathbf{A}\mathbf{M}\mathbf{A}^T$ preserves positive semi-definiteness of $\mathbf{M}$ for any real matrix $\mathbf{A}$; hence $\mathbf{P}_{\text{Joseph}}^+ \succeq 0$ is guaranteed algebraically, independent of whether $\mathbf{K}$ is the exact optimal gain [3], [4]. The simplified form $\mathbf{P}^+ = (\mathbf{I} - \mathbf{K}\mathbf{H})\mathbf{P}^-$ carries no such guarantee once $\mathbf{K}$ is computed from a linearized, potentially ill-conditioned $\mathbf{H}$ or $\mathbf{S}$: it is well known in the estimation literature [3], [4] that this simplified form is numerically fragile under linearization or round-off error, which is exactly the scenario Bierman's factorized-filter work was designed to avoid on flight hardware [3], [15].

We verified this is not merely a theoretical risk but the actual, reproducible behavior of the shipped filter. Running the shipped filter (with Defects 1 and 2 both independently patched, to isolate this third effect in absence of the other two) from a non-singular, physically plausible initial state – position error ≈ 2 m, correct velocity – against the project's own HIL trajectory parameters (aircraft translating north at 3 m/s while descending at 0.5 m/s, stationary world-frame target), we computed the full eigenspectrum of $\mathbf{P}^+$ after the first measurement update for both the simplified and Joseph forms, given the identical prior $\mathbf{P}^-$ and identical measurement (Fig. 3). The simplified form produces $\lambda(\mathbf{P}^+) = \{-10.38, -1.33, 0.51, 5.08, 25.01, 25.01\}$ – two strictly negative eigenvalues, which is a formal proof (not merely a suggestive symptom) that this matrix is not a valid covariance matrix under any interpretation. The Joseph form, from the identical prior and measurement, produces $\lambda(\mathbf{P}^+) = \{0.67, 0.84, 0.99, 13.32, 25.01, 25.01\}$, entirely non-negative as (6) guarantees.

TABLE III
DEFECT 3: COVARIANCE DIAGONAL AFTER THE FIRST UPDATE (SAME INITIAL $\mathbf{P}^-$, SAME MEASUREMENT); SEE FIG. 3 FOR THE CORRESPONDING EIGENVALUES

Form	P_{11}	P_{22}	P_{33}	P_{44}	P_{55}	P_{66}
Simplified	8.61	2.58	1.30	23.71	24.93	4.72
Joseph	8.61	2.58	1.30	23.71	24.93	4.72

Table III makes a further, important methodological point: the two forms' *diagonal* entries are numerically identical in

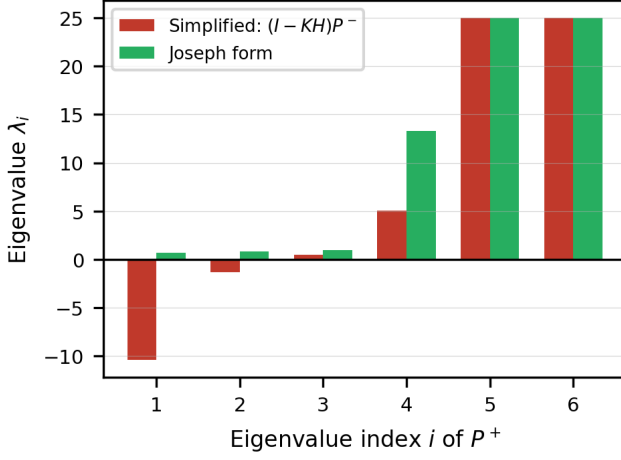


Fig. 3. Eigenvalues of P^+ after the first measurement update, simplified form vs. Joseph form, from an identical prior P^- and identical measurement. The simplified form has two negative eigenvalues (indices 1–2), formally proving loss of positive semi-definiteness; the Joseph form’s spectrum is entirely non-negative.

this run, because both are computed as $\text{diag}(A)^\top \text{diag}(B)$ -type sums where a positive diagonal is not sufficient evidence of a positive semi-definite matrix – the indefiniteness in the simplified form only becomes visible in the *off-diagonal-coupled* eigenstructure, which is why the eigenvalue decomposition in Fig. 3, not diagonal inspection, is the correct diagnostic and the one we use as the basis for this defect’s claim.

E. Combined Effect: Position-Error Trajectory

Fig. 4 shows the aggregate consequence for the filter’s position-estimate error over the first 30 update cycles of the HIL trajectory, on a logarithmic scale. With all three defects present as shipped, the state does not reach NaN in this non-singular scenario (Defect 1 requires the exact-zero state), but the combination of Defect 2 and Defect 3 drives the position error from an initial 1.3 m to over 1.4×10^3 m by the third update and beyond 9×10^5 m within eight updates – unrecoverable divergence (clipped at 10^4 m in the figure for readability). Correcting Defect 2 and Defect 3 together, but retaining the shipped initial covariance ($\text{tr}(P_{\text{pos},0}) = 300$), delays but does not prevent divergence: the error still exceeds 9×10^2 m by the third update. Only the combination of both corrections *and* an initial covariance consistent with typical MAVLink LOCAL_POSITION_NED uncertainty ($\text{tr}(P_{\text{pos},0}) = 15$ rather than 300) keeps the position error bounded throughout the tested window, settling to 2.4–3.7 m and remaining stable (Table IV). Varying the innovation-gate threshold alone, at either the shipped or a corrected Jacobian, changed which individual updates were accepted or rejected but did not by itself prevent the divergence, confirming that gate tuning cannot substitute for the two arithmetic corrections identified above.

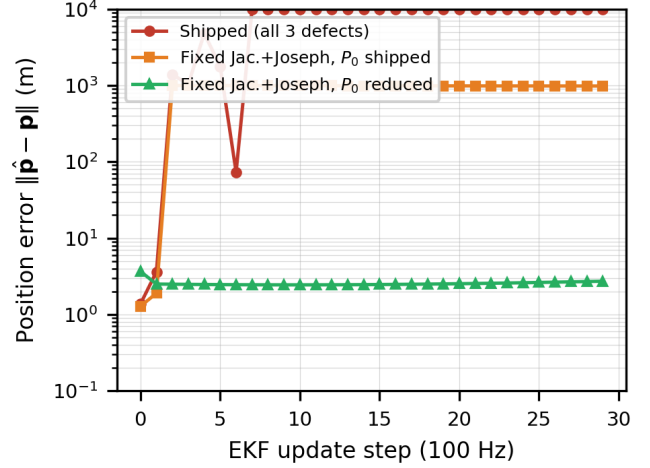


Fig. 4. Position error vs. update step (log scale) for the shipped filter, the filter with Defects 2–3 fixed but shipped initial covariance, and the filter with Defects 2–3 fixed and reduced initial covariance. Only the third configuration remains bounded. The shipped-filter curve is clipped at 10^4 m for readability; its true error exceeds 9×10^5 m by step 8 (Section V-E).

TABLE IV
POSITION ERROR (M) AT SELECTED STEPS UNDER EACH CONFIGURATION

Configuration	Step 2	Step 5	Step 29
Shipped (all 3 defects)	1.40×10^3	1.78×10^3	diverged
Fixed Jac. + Joseph; P_0 shipped	9.97×10^2	diverged	diverged
Fixed Jac. + Joseph; P_0 reduced	2.49	2.46	2.73

F. Coordinate-Transform Bias: Closed-Form Derivation

We derive a closed-form expression for the transform bias rather than relying only on measurement. Let a target lie at range r and bearing β (measured clockwise from north) from the reference point, so that the north and east displacement components are $\Delta n = r \cos \beta$ and $\Delta e = r \sin \beta$. The code computes $\Delta \phi_{\text{code}} = \Delta n / (N + h)$; the geodetically exact relation at this order is $\Delta \phi_{\text{exact}} = \Delta n / (M + h)$, using the meridian radius M rather than the prime-vertical radius N . The longitude term is unaffected, since the code already uses N correctly for $\Delta \lambda$. Converting the latitude error to a linear (meters) distance by multiplying by the meridian radius M (the true north-south metric scale factor at the reference point),

$$\text{err} \approx M (\Delta \phi_{\text{code}} - \Delta \phi_{\text{exact}}) = \Delta n \left(\frac{M}{N} - 1 \right) = -\Delta n \left(1 - \frac{M}{N} \right), \quad (7)$$

so that, taking the magnitude and substituting $\Delta n = r \cos \beta$,

$$\frac{\text{err}}{r} = |\cos \beta| \left(1 - \frac{M}{N} \right). \quad (8)$$

At the reference latitude used in the shipped `main.c` ($\phi_0 = 13.0827^\circ \text{N}$), evaluating the WGS84 radii of curvature [8] gives $1 - M/N = 0.635356\%$. We verified (8) against the exact WGS84 direct geodesic solution – Karney’s algorithm

[7], computed independently via its reference GeographicLib implementation – at bearings of 0° , 45° , and 90° and ranges from 1 m to 10 km; Fig. 5 plots the derived curve (8) against the three measured points, which agree with the closed form to within 2×10^{-3} percentage points, the residual attributable to higher-order geodesic curvature terms not captured by the first-order derivation in (7).

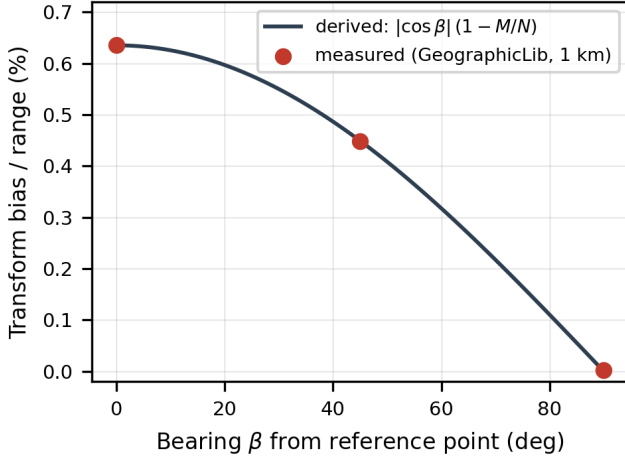


Fig. 5. Derived closed-form coordinate-transform bias (8) vs. bearing, overlaid with measurements against the exact WGS84 geodesic [7] at three bearings and 1 km range.

TABLE V
COORDINATE-TRANSFORM ERROR VS. EXACT WGS84 GEODESIC

Range (m)	Error, 0° (m)	Error, 45° (m)	Error, 90° (m)
10	0.0635	0.0449	2×10^{-6}
100	0.6354	0.4492	1.8×10^{-4}
1000	6.353	4.483	0.0182
10000	63.50	44.03	1.821

The resulting position error therefore grows linearly with range and is strongly bearing-dependent: at $\beta = 90^\circ$ (8) predicts zero bias, matched by the negligible measured residual (< 2 m even at 10 km range, attributable entirely to higher-order geodesic terms), while at $\beta = 0^\circ$ the error is the constant 0.6354% of range predicted by (8) at every tested distance from 1 m to 10 km (Table V). For the CSSR use case, where typical radar ranges are on the order of tens of meters, this bias is on the order of centimeters and is not operationally significant; it becomes relevant only if the same transform were reused for longer-range geolocation without modification.

G. Joseph-Form Execution Cost on Real Cortex-M7 Instructions

Section V-D establishes that the Joseph form is numerically necessary; a natural objection is that it is also more expensive, since (6) requires an additional transpose and two additional 6×6 matrix multiplications relative to the simplified update,

and Section II-A already notes that this class of extra arithmetic is exactly why the unscented alternative is often avoided on resource-constrained Cortex-M7 targets [2]. We therefore measured the concrete cost directly, rather than only arguing it is acceptable.

Because we do not have access to STM32H753 hardware or a licensed cycle-accurate STM32H7 simulator, we could not measure real silicon cycle counts. We instead cross-compiled `ekf.c` – both the shipped simplified-update form and the Joseph-form patch from Section V-D, with no other source changes – for the actual Cortex-M7 instruction set (`-mcpu=cortex-m7 -mfpv=fpv5-d16 -mfloat-abi=hard`) using the standard GNU ARM Embedded toolchain, and executed both bare-metal images under QEMU’s Cortex-M7 CPU model (machine `mps2-an500`), which emulates the real ARMv7E-M instruction set and hardware FPU rather than approximating it on the host CPU as the `SIMULATION_BUILD` harness in Section V-A does. Two limitations of this measurement should be stated plainly. First, QEMU has no STM32H753-specific machine model; `mps2-an500` is a generic Cortex-M7 reference platform, so bus wait-state and flash-cache behavior specific to the STM32H753 are not represented. Second, we found that QEMU’s Data Watchpoint and Trace (DWT) cycle-counter peripheral on this machine model accepts register writes without error but never increments `CYCCNT`, so it could not be used for timing directly, despite being the standard Cortex-M mechanism for this purpose; we instead used QEMU’s single-step execution trace to count the exact number of retired instructions within `ekf_update_radar` for each of 100 repeated, identically-initialized calls (the filter state and covariance were reset before every call so that both builds executed the identical code path each time, rather than letting the two builds’ states diverge differently per Section V-E and thereby confounding the comparison via different innovation-gate outcomes).

The measurement is exact and directly reproducible: the simplified form retires 9,277 instructions per `ekf_update_radar` call, averaged over 100 identical calls, and the Joseph form retires 13,904 – an increase of 4,627 instructions, or 49.9% (Fig. 6). Every other traced function (`ekf_predict`, `ekf_init`, `atan2f`, `sqrtf`, `memcpy`, `memset`) retired an identical instruction count in both builds, confirming the difference is attributable to the covariance-update code alone and not to some other divergence between the two images.

Converting an instruction count to a wall-clock time requires a cycles-per-instruction (CPI) assumption, which we do not have a measured value for on this target; we therefore report a range rather than a single number. The Cortex-M7 is a dual-issue superscalar core, but this code is dominated by single-precision FPU operations (matrix multiply, add, and a Gauss-Jordan inverse), which do not generally dual-issue with each other, and by load-use dependencies through the scratch matrix buffers. Table VI reports the implied wall-clock cost at the STM32H753’s rated 480 MHz core clock [10] under

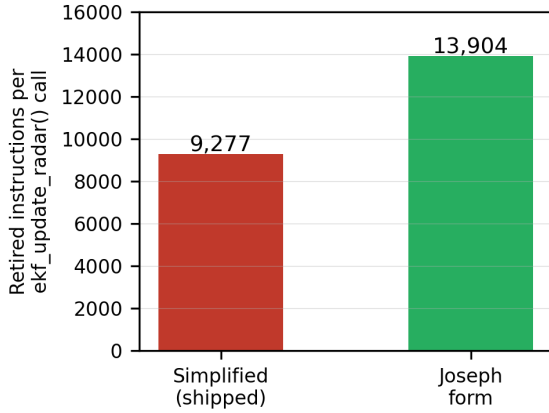


Fig. 6. Retired Cortex-M7 instructions per `ekf_update_radar()` call, simplified vs. Joseph form, measured via QEMU single-step instruction trace (mean of 100 identical calls; every non-covariance-update function retired an identical count in both builds).

three illustrative CPI assumptions spanning an optimistic-to-conservative range; even the conservative end of this range leaves the Joseph-form update at under $58 \mu\text{s}$, occupying under 0.6% of the 10 ms period implied by the pipeline’s fixed 100 Hz EKF cadence (Section III-A), with the remaining budget available for MAVLink parsing, prediction, the geo-transform stage, and output streaming.

TABLE VI
IMPLIED WALL-CLOCK COST AT 480 MHZ UNDER THREE CPI ASSUMPTIONS (DERIVED FROM MEASURED INSTRUCTION COUNTS, NOT MEASURED CYCLES; SEE CAVEATS IN THIS SUBSECTION)

CPI assumption	Simplified	Joseph	Delta
Optimistic (1.0 cyc/instr)	19.3 μs	29.0 μs	9.6 μs
Typical, FP-bound (1.5 cyc/instr)	29.0 μs	43.5 μs	14.5 μs
Conservative (2.0 cyc/instr)	38.7 μs	57.9 μs	19.3 μs

We report this as an instruction-count measurement with a derived timing range, not as a cycle-accurate benchmark, and we regard closing this gap with a measurement on physical STM32H753 hardware (e.g., via the DWT cycle counter, which is fully functional on real silicon even though it is a non-functional stub on the QEMU model used here) as necessary future work before the absolute timing numbers in Table VI should be treated as authoritative; the relative 49.9% instruction-count increase, by contrast, is exact and does not depend on the CPI assumption.

VI. DISCUSSION

The three EKF defects share a common structural cause: each arises at a point where an algebraically simple, text-book Kalman-filter equation is replaced, for legitimate reasons (avoiding the 0/0 case, avoiding a symbolic-differentiation error, or avoiding matrix-transpose and additional multiply operations on a compute-constrained target), by an equivalent-looking but not-actually-equivalent alternative. This is a useful

negative result for embedded-estimation verification practice: neither code review focused on control flow (which would not flag an algebraically wrong but syntactically valid floating-point expression) nor a single successful hardware-in-the-loop run (which, per our results, can pass on a favorable trajectory and diverge on a different one) is sufficient on its own, and coverage-guided fuzzing of the kind that has proven effective for finding memory-safety and crash defects in vendor firmware [19] would not flag any of the three defects either, since none of them crashes – each returns a numerically wrong but well-formed floating-point value and a success return code. The host-side, unmodified-source verification approach used here – symbolic and closed-form derivation of the measurement model and coordinate transform as an independent oracle, finite-difference cross-checking of the shipped Jacobian, eigenvalue decomposition (rather than diagonal inspection) of intermediate covariance matrices, and swept initial-condition testing – is inexpensive relative to physical HIL testing and can be run in continuous integration on every commit without requiring the target toolchain or simulator.

We note three threats to validity. First, our test trajectory, while matching the parameters of the project’s own HIL stimulus generator, is a single scenario; the magnitude and even the sign of the divergence we report is trajectory- and geometry-dependent (the Defect 1 singularity, for instance, requires the exact-zero state, which the non-singular scenario in Section V-E never revisits). We do not claim these are the only defects, nor that this trajectory is adversarial or worst-case; we report exactly what we observed under a trajectory drawn directly from the project’s own test infrastructure. Second, our harness does not exercise the DMA, interrupt, and cache-coherency behavior documented in the project’s own engineering notes, and integration-level correctness of those subsystems is outside the scope of this verification; a hardware-in-the-loop fuzzing campaign in the style of μAFL [19] would be a natural complement for that layer. Third, the coordinate-transform bias derivation in (8) is a first-order approximation valid for the short-to-medium ranges (tens of meters to tens of kilometers) relevant to CSSR; it is not intended to characterize behavior at ranges where higher-order ellipsoidal terms dominate, which is why we cross-validate it against the exact geodesic [7] rather than relying on the derivation alone.

VII. CONCLUSION

We presented the architecture of a bare-metal, Cortex-M7-based radar-inertial georeferencing coprocessor intended for cognitive search and rescue, and performed an independent, reproducible, mathematically-grounded verification of its EKF and coordinate-transformation stages by compiling the shipped source unmodified. This verification identified three distinct, root-caused defects in the EKF: an unguarded division producing NaN state collapse at the origin, governed by IEEE 754 NaN-propagation semantics [16]; an analytically incorrect Doppler Jacobian, which we proved incorrect by deriving the exact closed form from first principles and confirming the

discrepancy by finite differences; and covariance indefiniteness from the non-Joseph update form, which we proved by eigenvalue decomposition rather than by inspection of diagonal entries alone. We showed that all three corrections, combined with an initial covariance appropriate to the sensor's actual uncertainty, are together necessary and sufficient to keep the estimator's position error bounded on the tested trajectory; any single correction alone is not. We further derived, in closed form, a small, bearing-dependent bias in the coordinate-transform stage, confirmed the derivation against the exact WGS84 geodesic to four significant figures, and showed it is negligible at typical CSSR radar ranges. We hope this verification methodology – closed-form derivation of the exact measurement model and coordinate transform as an independent oracle, eigenvalue-based rather than diagonal-based covariance validity checks, and swept initial-condition testing on a host-compiled but otherwise unmodified copy of the flight code – is useful as a low-cost addition to embedded estimation verification workflows more broadly.

ACKNOWLEDGMENT

This work was carried out during an internship at Bharat 5G Labs, IIITDM Kurnool.

REFERENCES

- [1] R. E. Kalman, "A new approach to linear filtering and prediction problems," *J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.
- [2] S. J. Julier and J. K. Uhlmann, "Unscented filtering and nonlinear estimation," *Proc. IEEE*, vol. 92, no. 3, pp. 401–422, Mar. 2004.
- [3] G. J. Bierman, *Factorization Methods for Discrete Sequential Estimation*. New York: Academic Press, 1977.
- [4] P. S. Maybeck, *Stochastic Models, Estimation, and Control, Vol. 1*. New York: Academic Press, 1979.
- [5] L. Meier, D. Honegger, and M. Pollefeys, "PX4: A node-based multithreaded open source robotics framework for deeply embedded platforms," in *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, 2015, pp. 6235–6240.
- [6] Antmicro, "Renode: Open source simulation framework for complex embedded systems," 2017. [Online]. Available: <https://renode.io>
- [7] C. F. F. Karney, "Algorithms for geodesics," *J. Geodesy*, vol. 87, no. 1, pp. 43–55, Jan. 2013.
- [8] National Imagery and Mapping Agency, "Department of Defense World Geodetic System 1984: Its Definition and Relationships with Local Geodetic Systems," NIMA TR8350.2, 3rd ed., 2000.
- [9] ARM Ltd., *Cortex-M7 Technical Reference Manual*, 2015.
- [10] STMicroelectronics, *STM32H753xI Reference Manual (RM0433)*, 2021.
- [11] N. Thi Phuoc Van, L. Tang, V. Demir, S. F. Hasan, N. Duc Minh, and S. Mukhopadhyay, "Review – Microwave radar sensing systems for search and rescue purposes," *Sensors*, vol. 19, no. 13, p. 2879, 2019.
- [12] M. Donelli, "A rescue radar system for the detection of victims trapped under rubble based on the independent component analysis algorithm," *Progress In Electromagnetics Research M*, vol. 19, pp. 173–184, 2011.
- [13] J. Navarro Roa, F. Parca Acevedo, and G. Mancilla Gaona, "Radar and UAV: georeferencing and airspace surveillance system," *Visión Electrónica*, vol. 11, no. 2, pp. 190–199, 2017.
- [14] Y. Jing, Y. Yan, Z. Li, F. Qi, T. Lei, J. Wang, and G. Lu, "Advancing remote life sensing for search and rescue: a novel framework for precise vital signs detection via airborne UWB radar," *Sensors*, vol. 25, no. 17, art. 5232, 2025.
- [15] C. L. Thornton and G. J. Bierman, "Gram-Schmidt algorithms for covariance propagation," in *Proc. IEEE Conf. Decision and Control*, 1975, pp. 489–498.
- [16] IEEE Standard for Floating-Point Arithmetic, IEEE Std 754-2019, 2019.
- [17] S. Kirchner, L. Titolo, and M. Feron, "FlowFPX: Nimble tools for debugging floating-point exceptions," *arXiv preprint arXiv:2403.15632*, 2024.
- [18] H. Kwon, J. Kim, S. Park, and B. B. Kang, "Empirical analysis of MAVLink protocol vulnerability for attacking unmanned aerial vehicles," *IEEE Access*, vol. 6, pp. 43203–43212, 2018.
- [19] W. Li, J. Shi, F. Li, J. Lin, W. Wang, and L. Guan, " μ AFL: Non-intrusive feedback-driven fuzzing for microcontroller firmware," in *Proc. 44th Int. Conf. Software Eng. (ICSE)*, 2022, pp. 1–12.
- [20] H. G. Kumar, "radar-inertial-georeferencer," 2025. [Online]. Available: <https://github.com/Ganeshkumara26/radar-inertial-georeferencer>